
QUEEN DOMINATION BY SAT SOLVING +

Implementation available at: <https://github.com/TahaRostami/Gamma/tree/gamma-plus>

Introduction

Given an $n \times n$ chessboard, the queen domination problem asks for the minimum number of queens required to dominate all squares of the board. A queen dominates every square lying in the same row, column, diagonal, or anti-diagonal as its position. The corresponding minimum number of queens is denoted by $\gamma(Q_n)$. Several theoretical results provide useful bounds on $\gamma(Q_n)$ and substantially reduce the search space. In particular, the inequality $\gamma(Q_n) - \gamma(Q_{n-1}) \leq 1$ limits the rate at which the domination number can grow, while the lower bound $\gamma(Q_n) \geq \lceil (n-1)/2 \rceil$ further restricts the range of feasible values [1]. Together, these bounds often reduce the number of candidate values for $\gamma(Q_n)$ to only one or two for a given board size.

Despite these results, determining the exact domination number or enumerating all optimal solutions up to isomorphism generally requires computational methods. Most approaches proposed in the literature rely on highly specialized search procedures designed specifically for the combinatorial structure of the problem [2, 3, 4]. More recently, however, it was shown that carefully engineered SAT encodings can achieve performance competitive with specialized solvers for queen domination while additionally providing proof certificates, thereby enabling independently verifiable and potentially more trustworthy results [5]. In particular, the SAT-based encoding of [5] combined literal ordering heuristics, symmetry breaking constraints, and Cube-and-Conquer techniques to obtain strong computational performance, correct previously reported erroneous results, and resolve a previously open instance in the model enumeration setting.

Motivated by these developments, this work investigates a new SAT formulation for the queen domination problem that significantly outperforms the previous SAT-based approach. The formulation extends the encoding of [5] by introducing auxiliary variables associated with board lines, namely rows, columns, diagonals, and anti-diagonals. These variables expose additional geometric structure to the SAT solver and enable stronger forms of global reasoning during search.

The central observation underlying the formulation is straightforward: each queen activates exactly one row, one column, one diagonal, and one anti-diagonal. Consequently, any placement using at most γ queens can activate at most 4γ lines in total. This makes it possible to impose an additional cardinality constraint over the line variables, which in practice turns out to be one of the primary sources of the observed performance improvement.

The encoding further incorporates several optimizations, including Hilbert curve variable ordering, static symmetry breaking constraints, and a Cube-and-Conquer strategy tailored to totalizer-based cardinality encodings. In the model enumeration setting, previous work first enumerated all solutions and then performed a separate final UNSAT verification step [5]. In contrast, the present approach employs `cadical-exhaust`¹ together with `drat-trim-t`², allowing solving and proof verification to be performed in a combined step.

Encoding Queen Domination with Line Variables

We consider the queen graph of an $n \times n$ chessboard, where each square corresponds to a vertex and edges connect squares that share a row, column, diagonal, or anti-diagonal. The objective is to determine whether there exists a dominating set of size at most γ .

Our formulation extends the encoding introduced in previous work [5] by introducing auxiliary variables corresponding to board lines. This additional structure enables the SAT solver to reason more globally about the geometry of the chessboard.

Let Q_i denote a Boolean variable indicating whether a queen is placed on square i , assuming squares are indexed from 1 to n^2 in row-major order. In addition, for every row, column, diagonal, and anti-diagonal ℓ , we introduce a Boolean variable L_ℓ , indicating whether line ℓ is active, i.e., whether it contains at least one queen. Furthermore, for each square i , let $\mathcal{L}(i)$ denote the set of the four lines passing through square i .

¹<https://github.com/curtisbright/cadical-exhaust>

²<https://github.com/curtisbright/drat-trim-t>

The domination constraints are expressed using the line variables. Indeed, every square must be dominated, and a square i is dominated if at least one of its incident lines is active. Thus, for every square i , we add the clause

$$\bigvee_{\ell \in \mathcal{L}(i)} L_\ell.$$

Next, if a line ℓ is active, then by definition at least one queen must lie on that line. Therefore, for every line ℓ , we add

$$\neg L_\ell \vee \bigvee_{i \in \ell} Q_i.$$

Since the objective is to determine whether there exists a dominating set of size at most γ , the encoding includes a cardinality constraint limiting the number of queens:

$$\sum_{i=1}^{n^2} Q_i \leq \gamma.$$

This constraint is encoded using a totalizer encoding. As observed in prior work [6], the ordering of literals significantly impacts solver performance. To preserve geometric locality, we order the queen variables according to a Hilbert curve traversal of the chessboard, following the approach introduced in previous work [5]. This ordering better reflects the board structure and improves propagation behavior in practice.

In addition to introducing the line variables, a major source of the performance improvement comes from imposing a second cardinality constraint on these variables. Since each queen lies on exactly one row, one column, one diagonal, and one anti-diagonal, a queen can activate at most four lines. Consequently, any solution using at most γ queens can activate at most 4γ lines. We therefore impose the constraint

$$\sum_{\ell \in \mathcal{L}} L_\ell \leq 4\gamma.$$

For this constraint, the line variables are ordered by decreasing line length before being passed to the mTotalizer cardinality encoder.

Symmetry Breaking

To reduce redundant exploration of symmetric solutions, we incorporate static symmetry-breaking constraints, similarly to previous work [5]. Our formulation uses a lexicographic symmetry-breaking scheme based on the encoding proposed by Harvey.

Apart from the identity transformation, the $n \times n$ chessboard has seven geometric symmetries: rotations by 90° , 180° , and 270° , as well as reflections across the horizontal axis, vertical axis, main diagonal, and anti-diagonal. Let X denote the Boolean vector associated with the original ordering of the queen variables. For each non-identity symmetry, we construct a transformed vector Y and enforce the lexicographic condition

$$X \leq Y.$$

To encode the lexicographic comparison

$$[x_1, \dots, x_N] \leq [y_1, \dots, y_N],$$

we introduce auxiliary variables $a_0, a_1, \dots, a_N$, where $N = n^2$. Intuitively, each variable a_i captures whether the suffix

$$[x_{i+1}, \dots, x_N]$$

is lexicographically smaller than or equal to the corresponding suffix

$$[y_{i+1}, \dots, y_N].$$

Following Harvey's construction, the encoding introduces the ternary clauses

$$\begin{aligned} a_{i+1} \vee y_{i+1} \vee \neg a_i, \\ a_{i+1} \vee \neg x_{i+1} \vee \neg a_i, \end{aligned}$$

and

$$y_{i+1} \vee \neg x_{i+1} \vee \neg a_i,$$

for every $0 \leq i < N$, together with unit clauses enforcing a_0 and a_N to be true. These constraints are generated for all seven nontrivial symmetries of the chessboard. In contrast to previous work, we additionally observed that arranging the symmetry-breaking vectors according to a Hilbert curve ordering further improves solver performance in practice.

Cube-and-Conquer

To solve larger instances more effectively, we use the Cube-and-Conquer paradigm, which partitions the original SAT instance into many smaller subproblems, called cubes, that can then be solved independently by a conflict-driven SAT solver. For smaller instances, Cube-and-Conquer is often optional and may provide only moderate speedups. For larger instances, however, partitioning becomes essential, since solving the original instance directly becomes increasingly difficult.

Previous work [5] employed a Cube-and-Conquer approach based on `march_cu`. Although effective on smaller instances, that framework struggled to scale on larger queen domination instances. In the present work, we instead adopt a more scalable Cube-and-Conquer framework [7], which is designed specifically for totalizer-based encodings. For this reason, we encode the queen cardinality constraint using a standard totalizer rather than the `mTotalizer` encoding used previously.

In particular, our cubing strategy selects auxiliary variables from the totalizer encoding of the queen cardinality constraint. These variables encode partial counting information, namely whether at least a certain number of queen variables are assigned true within a sub-block of the ordered variables. Branching on such variables partitions the search space according to meaningful cardinality structure rather than only individual queen placements, producing independent subproblems that can then be solved separately. Interested readers can find a detailed explanation of this procedure in [7].

Results

Comparison with the Previous State-of-the-Art

Table 1: Runtime comparison, in seconds, between the previous SAT encoding [5] and the proposed line-based encoding on UNSAT instances. For each board size n , the solver was executed with parameter $\gamma - 1$, corresponding to the setting used to establish lower bounds on the domination number. Symmetry breaking and Cube-and-Conquer were enabled in both approaches. These experiments evaluate the effectiveness of the encoding in the optimization setting, where the goal is to determine the optimal value of $\gamma(Q_n)$. All runtimes are reported in seconds.

n	Previous SAT Approach [5]	New SAT Approach
12	0.47	0.17
13	5.00	1.82
14	36.92	10.17
15	1577.32	139.00

Table 2: Runtime comparison for model enumeration with solving and verification. The previous SAT approach [5] used the earlier encoding, first enumerated all solutions, and then performed a separate UNSAT verification step. In contrast, the new approach uses the proposed line-based encoding together with `cadical-exhaust` and `drat-trim-t`, allowing solving and proof verification to be performed in a combined step. Symmetry breaking and Cube-and-Conquer were enabled in both approaches. All runtimes are reported in seconds.

n	Previous SAT Approach [5]	New SAT Approach
12	27	4
13	513	101
14	20,531	1,760
15	416,789	128,956

Observations and New Results

For brevity, we do not report the full experimental results here, but summarize the main observations. Throughout the experiments, we distinguish between two encodings: the previous SAT encoding [5] and the new line-based encoding proposed in this work. We also distinguish between two computational settings: determining the optimal value of $\gamma(Q_n)$, and model enumeration up to isomorphism.

The proposed line-based formulation naturally admits both a SAT encoding and a corresponding integer programming (IP) formulation. Across all experiments, we observed that the new encoding significantly outperforms the previous encoding, regardless of whether SAT solvers or IP solvers are used.

We further observed that SAT solvers substantially outperform IP solvers in the model enumeration setting, with the best overall performance achieved by SAT solving using the proposed line-based encoding. In contrast, for the optimization setting—where the goal is to determine the optimal value of $\gamma(Q_n)$ —modern IP solvers often outperform SAT solvers on larger board sizes when combined with the new encoding.

Using these observations, we resolved a previously open instance in the optimization setting. In particular, using the line-based IP formulation together with Gurobi, we established in only 5,162 seconds that $\gamma(Q_{26}) = 14$.

Related Work

Several computational approaches to the queen domination problem have relied on specialized search procedures tailored to the structure of the problem. Gibbons and Webb [3] incorporated isomorphism rejection into a backtracking search, while Kearse and Gibbons [4] explored several variants of backtracking based on queen placements, undominated squares, and attacking sets. More recently, Bird developed the highly optimized tool UNIDOM, which uses problem-specific branching heuristics, bounding rules, specialized data structures, and distributed search to solve several open queen domination instances [2]. These approaches are highly customized to queen domination, whereas the approach in this work is based on encoding the problem into standard SAT and IP formulations. Despite relying on more general-purpose solving frameworks, the proposed encoding achieves performance superior to these specialized approaches.

Recent work demonstrated that Hilbert curve variable ordering can significantly improve SAT encodings for the queen domination problem when combined with symmetry breaking and Cube-and-Conquer techniques [5]. The present work builds upon and extends several of these ideas. In particular, we preserve the Hilbert curve ordering for the queen variables, but substantially modify the underlying encoding. Rather than encoding domination relationships directly between queens and squares, we introduce auxiliary variables associated with rows, columns, diagonals, and anti-diagonals, and express domination through these line variables.

We additionally improve the symmetry-breaking component by arranging the symmetry vectors according to a Hilbert curve ordering, whereas previous work used the default variable ordering. Furthermore, we adopt and implement a more recent and scalable Cube-and-Conquer framework tailored to totalizer-based encodings [7], replacing the earlier framework used in [5]. Finally, in the model enumeration setting, previous work first enumerated all solutions and only afterwards performed proof generation and verification on the final UNSAT instance. In contrast, the present approach combines solving, proof generation, and verification into a unified workflow.

Recent work has also investigated relaxations of the queen domination problem using line-level abstractions to improve lower bounds and simplify proofs [8]. The primary objective of that work was theoretical in nature, namely to introduce a relaxed formulation that is potentially easier to analyze and reason about mathematically. In contrast, our formulation solves the original queen domination problem exactly rather than a relaxation. Another important distinction is that previous line-based relaxations treated rows, columns, diagonals, and anti-diagonals separately, whereas our formulation treats all line types uniformly and imposes a single global bound of 4γ on the total number of active lines. Empirically, we found this unified approach to perform best in both SAT and integer programming settings.

Table 3: Experimental results obtained using Slurm job arrays on the Iris HPC cluster at the University of Luxembourg. These experiments were run separately from the other reported experiments on regular Iris Intel Xeon CPU nodes, equipped with either $2 \times$ Intel Xeon E5-2680 v4 @ 2.4 GHz or $2 \times$ Intel Xeon Gold 6132 @ 2.6 GHz processors.

n	γ	# cubes	# Models	Solve Time (s)	Verification (s)	Proof Size
3	1	$2^1 (= 2)$	1	0.02	0.15	1.24 KB
4	2	$2^1 (= 2)$	3	0.02	0.15	2.35 KB
5	3	$2^1 (= 2)$	37	0.03	0.16	10 KB
6	3	$2^1 (= 2)$	1	0.03	0.15	14 KB
7	4	$2^1 (= 2)$	13	0.15	0.15	128 KB
8	5	$2^1 (= 2)$	638	0.41	0.34	565 KB
9	5	$2^1 (= 2)$	21	0.85	0.65	1.43 MB
10	5	$2^1 (= 2)$	1	0.44	0.38	817 KB
11	5	$2^1 (= 2)$	1	0.77	0.57	1.52 MB
12	6	$2^5 (= 32)$	1	2.38	3.75	4.43 MB
13	7	$2^5 (= 32)$	41	50	45	96.5 MB
14	8	$2^{14} (= 16,384)$	588	599	1,956	780 MB
15	9	$2^{15} (= 32,768)$	25,872	14,761	20,847	13.3 GB
16	9	$2^{15} (= 32,768)$	371	1,688	4,052	2 GB
17	9	$2^{15} (= 32,768)$	22	8,919	10,969	10.4 GB
18	9	$2^{15} (= 32,768)$	2	5,051	8,040	5.9 GB
19	10	$2^{15} (= 32,768)$	11	171,575	252,706	86.5 GB

References

- [1] J.T. Hedetniemi and S.T. Hedetniemi. Domination in chessboards. In *Structures of Domination in Graphs*, pages 341–386. 2021.
- [2] W.H. Bird. *Computational methods for domination problems*. PhD thesis, University of Victoria, 2017.
- [3] P.B. Gibbons and J.A. Webb. Some new results for the queens domination problem. *Australasian Journal of Combinatorics*, 15:145–160, 1997.
- [4] M.D. Kearse and P.B. Gibbons. Computational methods and new results for chessboard problems. Technical Report 144, CDMTCS Research Report Series, 2000.
- [5] Taha Rostami and Curtis Bright. Queen domination by sat solving. *arXiv preprint arXiv:2508.11945*, 2025.
- [6] J.E. Reeves, J. Filipe, M.C. Hsu, R. Martins, and M.J.H. Heule. The impact of literal sorting on cardinality constraint encoding. In *Proc. AAAI Conf. on Artificial Intelligence*, volume 39, pages 11327–11335, 2025.
- [7] Z. Battleman, J. E. Reeves, and M. J. H. Heule. Problem partitioning via proof prefixes. In *Proceedings of the 28th International Conference on Theory and Applications of Satisfiability Testing (SAT 2025)*, pages 3:1–3:18. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2025.
- [8] A. Karandikar and A. Dutta. Improved lower bounds for queen’s domination via an exactly-solvable relaxation. *arXiv preprint arXiv:2304.06620*, 2023.